

FASTQ → QC → Alignment → Sorting/Indexing → QC (post-align) → Counting → DE → Enrichment

RNA-seq quantifies the transcriptome by converting RNA → cDNA and sequencing with standard Illumina chemistry. Design choices profoundly affect what you can detect.

WES captures the ~1–2% of the genome encoding protein-coding exons (~50 Mb), dramatically reducing sequencing cost relative to WGS while retaining 85% of known disease-associated variants.

- Biotin-labelled RNA baits (e.g. Agilent SureSelect, IDT xGen, Twist) hybridise to library fragments
- Streptavidin bead pull-down → wash off off-target fragments → elute, amplify, sequence
- Key parameter-On-target Rate: % of reads mapping inside the capture regions (expect ≥80%)
- Key parameter-Uniformity: % of target bases covered at ≥20% of mean depth (expect ≥95%)
- Key parameter-Mean Target Coverage: typically 100× for germline; 200–500× for somatic

The quality score Q encodes the probability P that the base call is WRONG:

FastQC results

- General Statistics table: total reads, % duplicates, % GC, % Q30-scan for outliers
- Per-base sequence quality heatmap-compare R1 vs R2, compare samples
- Per-sequence GC content-shift right = contamination; double peak = two species
- Sequence duplication levels-high % not always bad (amplicon, ChIP-seq)
- Adapter content-which adapters, how far from 3' end
- Box-whisker plot per cycle position across all reads
- Y-axis: Phred Q score (0–40); X-axis: base position in read
- Green zone (Q28–40): excellent; yellow zone (Q20–28): reasonable; red zone (Q20): poor
- Normal pattern: high quality at 5', gradual decline toward 3' (phasing accumulates)
- Sudden quality drop at specific position → flow cell bubble, cluster density spike, or wash failure
- Decision rule: if median drops below Q20 before 80% of read length → trim 3' end
- Histogram of mean quality per read across the run
- Ideal: single sharp peak at Q35–40

- Bimodal distribution → two populations (contamination, pooled samples, barcode bleed)
- Left-shifted peak → systemic quality issue (library prep failure, reagent degradation)
- Line plot: % A, T, G, C at each cycle-ideally flat and parallel (equal representation)
- First 10–15 bases often show composition bias (random hexamer priming in RNA-seq)-expected WARN/FAIL
- Dramatic divergence in middle of read → contamination with another library type
- For amplicon sequencing: expect non-flat composition by design-disable or ignore this module
- Observed GC distribution (blue) vs. theoretical normal distribution (red) for organism
- **Organism baseline: human ~50%, plant ~35–40%, bacteria varies widely**
- Narrow sharp peak above normal → PCR over-amplification of specific GC fraction
- Broad or right-shifted curve → ribosomal RNA contamination (esp. in poly-A selected RNA-seq)
- Multi-modal → contamination from another organism or sample mix-up
- % of N (undetermined base) per position
- Expect 0.1% throughout; any spike at early positions → base caller failure at start of run
- High N at end of reads → read length exceeded fragment length (sequencing into adapter)
- Most Illumina runs: single sharp bar at expected read length (e.g. 150 bp)
- Variable length distribution (SmartSeq, ONT) is expected-do not trim without reason
- Trimmed data: distribution shifts left-verify minimum length cutoff is reasonable
- FastQC samples first 200k reads and extrapolates duplication rate
- Unique reads (de-duplicated) shown in blue; total sequences in red
- WGS/WES: expect 30% duplication at normal coverage depths
- RNA-seq: expect higher duplication for highly expressed genes (biological, not PCR)
- ChIP-seq/ATAC-seq: WARN/FAIL is normal due to enrichment of narrow peaks
- PCR duplication: same fragment sequenced multiple times-remove with samtools markdup / Picard
- Optical duplication: physically adjacent clusters on patterned flow cell-remove with --OPTICAL_DUPLICATE_PIXEL_DISTANCE 2500 for patterned flow cells
- Any single sequence 0.1% of total reads is flagged
- FastQC attempts to match against a built-in contaminant DB (adapters, rRNA, PhiX)
- Unidentified overrepresented sequences: BLAST against NCBI to identify-often rRNA, ligation artifacts, or low-complexity repeats
- Per-position plot showing % reads containing each known adapter

- FastQC checks: Illumina Universal, Small RNA 3' Adapter, Nextera Transposase, SOLID Small RNA
- Adapter read-through: occurs when fragment read length → sequencer reads into adapter at 3' end
- Adapter contamination at 3' end: trim with Trimmomatic (ILLUMINACLIP), cutadapt, or fastp
- Adapter at 5' end: unusual-suggests library prep error or incorrect adapter orientation
-

Mature mRNA has had introns removed by splicing. If you align a 150 bp read that spans an exon-exon junction against a genomic reference, the read will appear to have a gap of thousands of bases in the middle.

A standard DNA aligner (BWA) would either:

- Reject the read as unmappable
- Map it to the wrong location in the genome
- Truncate one end

A **splice-aware aligner** (STAR, HISAT2) must:

1. Detect that the read spans a junction
2. Split the read at the correct point
3. Map each part to the correct exon
4. Confirm the junction against annotated or novel splice sites

How STAR works

STAR (Spliced Transcripts Alignment to a Reference) uses a two-pass alignment strategy and a suffix array index.

Seed search: STAR takes the read sequence and searches for the longest possible exact match (Maximal Mappable Prefix, MMP) in the genome suffix array. When the MMP hits a boundary (intron/mismatch), it records the anchor and searches for the next seed from the remaining unmapped portion of the read.

Stitching: Seeds are stitched together. If seeds from R1 land on exon 3 and exon 4 of the same gene with a gap matching a known intron -the read is assigned as a spliced alignment. STAR outputs this as a CIGAR string: e.g. 75M2691N75M (75 bases match, 2691 bp intron skipped, 75 bases match).

Two-pass mode (optional, --twopassMode Basic):

- Pass 1: align all samples, collect novel junctions discovered in the data
- Pass 2: re-align with novel junctions added to the index
- Improves alignment at novel or lowly-expressed splice sites
- Not used in most standard DE pipelines; useful for splicing analysis or low-coverage samples

Index build parameters

```
STAR \
  --runMode genomeGenerate \
  --genomeDir $STAR_INDEX \
  --genomeFastaFiles $GENOME \
  --sjdbGTFfile $GTF \
  --sjdbOverhang 149 \
  --runThreadN $THREADS \
  --genomeSAindexNbases 14
```

--runMode genomeGenerate Tells STAR this is an index-building run, not an alignment run. Produces binary index files in --genomeDir . these are reused for all subsequent alignments.

--sjdbGTFfile Loads splice junction database from the GTF at index time. STAR reads every annotated exon boundary and pre-encodes these as valid junction positions in the suffix array. Reads crossing these junctions will align correctly even if the junction spans tens of thousands of bases of intronic sequence.

--sjdbOverhang 149 = read length – 1. This defines the size of the sequence window extracted on each side of a splice junction for the junction database. The ideal value is read_length – 1 because the maximum overhang a read can have on one side of a junction is when it spans the junction with just 1 bp on the other side. Your reads are 150 bp → 149.

If you use the wrong value (e.g. 99 for 150 bp reads), junction detection still works but is suboptimal -reads that only slightly overlap a junction may not be assigned correctly.

--genomeSAindexNbases 14 Controls the length of the indexed string in the suffix array. The recommended formula is $\min(14, \log_2(\text{GenomeLength})/2 - 1)$. For mouse (~2.6 Gb): $\log_2(2.6 \times 10^9)/2 - 1 \approx 14.6 \rightarrow 14$.

Using a value too high = index build fails or very slow. Too low = many false positive seeds, slower alignment. For small genomes (< 10 Mb), this must be reduced (e.g. 11 for a 10 Mb genome).

Parameter explanations:

--readFilesCommand zcat -input files are gzipped; STAR cannot read gz directly without this flag. zcat decompresses on-the-fly via a pipe.

--outSAMtype BAM SortedByCoordinate - outputs a coordinate-sorted BAM directly, skipping the intermediate step of sorting with samtools sort. Sorted BAM is required by featureCounts and samtools index.

--outSAMattributes NH HI AS NM MD

- NH: number of loci the read maps to (1 = uniquely mapped)
- HI: hit index for multi-mappers (which of the NH alignments this is)
- AS: alignment score
- NM: number of mismatches
- MD: mismatch string (encodes which bases are mismatched) -needed for variant-aware analysis downstream

--outFilterMultimapNmax 20 -reads mapping to >20 genomic locations are discarded. Multi-mappers arise from gene families (e.g. ribosomal proteins, histones) and pseudogenes. 20 is the ENCODE standard. Reduce to 1 for uniquely-mapped-only analysis.

--alignSJoverhangMin 8 -minimum unannotated junction overhang: a read split at a novel junction must have at least 8 bp on each side. Prevents spurious novel junction calls from short overhangs.

--alignSJDBoverhangMin 1 -for annotated junctions, just 1 bp overhang is sufficient (since the junction is already trusted from the GTF).

--outFilterMismatchNoverReadLmax 0.04 -maximum mismatch rate = 4% of read length. For a 150 bp read, this allows up to 6 mismatches. Prevents highly divergent reads from mapping.

--alignIntronMin 20 / --alignIntronMax 1000000 -valid intron size range in bp. Mouse introns range from ~60 bp to >500 kb. The 1 Mb max catches very large introns in genes like Dystrophin.

- Genome loading -reading index into RAM (~30 GB for mouse; check available memory)
- Mapping speed -reads per second; typical 50–200k reads/sec with 8 threads
- Number of input reads -sanity check vs. FastQC read count
- Uniquely mapped reads % -target >75% for good mouse RNA-seq; <60% = problem
- % mapped to multiple loci -expected 5–15%; very high = contamination or wrong reference

Gene annotation

The GTF file defines the coordinate space for counting. For counting purposes, what matters is:

Gene model: which genomic coordinates belong to which gene. A gene typically has multiple transcripts; each transcript has multiple exons. For read counting:

- **Gene-level counting (featureCounts):** a read counts for a gene if it overlaps any exon of any transcript of that gene
- **Transcript-level counting (Salmon/kallisto):** reads are assigned probabilistically to individual transcripts using an EM algorithm, then optionally summarised to gene level

Annotation version matters: using GENCODE vM38 for alignment but vM39 for counting introduces coordinate mismatches. Always use the same GTF version throughout the pipeline.

Read counting

A sequenced read must be assigned to a feature (gene/transcript). Three complications:

1. Ambiguous reads (overlap multiple genes) featureCounts default: discard. Can enable --fraction to assign fractional counts to all overlapping genes. For most DE analysis, discarding is conservative and acceptable.

2. Multi-mapping reads (map to multiple genomic locations) featureCounts default: discard (count only uniquely mapped, NH=1). Enable --countMultiMappingReads to include. Inclusion increases counts for gene families (histones, ribosomal proteins) but risks double-counting.

3. Strandedness RNA-seq libraries are typically stranded. If you tell featureCounts the wrong strand:

- Unstranded when actually stranded: reads from antisense strand counted to wrong gene
- Forward when actually reverse: ~0 counts for everything

```
infer_experiment.py -r annotation.bed -i sample.bam
```

```
# Output: "Fraction of reads explained by '1++,1--,2+-,2-+'"
```

```
# ~50/50 = unstranded; ~90/10 = stranded
```

Pass to featureCounts: -s 0 (unstranded), -s 1 (forward), -s 2 (reverse/typical TruSeq stranded).

featureCounts -key parameters

```
featureCounts \  
-a $GTF \  
-o counts/count_matrix.txt \  
-T $THREADS \  
-p \          # paired-end  
--countReadPairs \ # count fragments, not reads  
-s 2 \        # reverse stranded (typical TruSeq)  
-t exon \     # feature type to count (GTF column 3)  
-g gene_id \  # attribute to group features by (GTF column 9)  
--fraction \  # (optional) fractional counting for multi-mappers  
$BAM_LIST
```

-t exon -only intervals annotated as "exon" in GTF column 3 are used as counting windows. Reads overlapping introns are ignored.

-g gene_id -reads overlapping the same gene_id across different exons are summed into one count per gene. Using transcript_id would give per-transcript counts.

-p --countReadPairs -for paired-end data, count the fragment (read pair) once, not each read separately. Without this, each read is counted independently → approximately 2× inflated counts.

Output: a tab-delimited matrix with GeneID, Chr, Start, End, Strand, Length, then one column per BAM file. The Length column = total exonic length of the gene (union of all exon intervals) -used for RPKM/FPKM normalisation.

mapping statistics

Metric	Expected	Flag if
Uniquely mapped reads %	>75%	<60%
% reads mapped to multiple loci	5–15%	>25%
% reads unmapped: too short	<5%	>20%
% reads unmapped: too many mismatches	<2%	>10%
% chimeric reads	<1%	>5%

From samtools flagstat:

```
for bam in ${OUTDIR}/*.bam; do  
  echo "==> $bam"  
  samtools flagstat -@ 4 $bam  
done
```

Fields to interpret:

- properly paired -both mates mapped in expected orientation and insert size; expect >90%
- singletons -one mate mapped, one did not; <5% typical
- with itself and mate mapped -both mates mapped but not necessarily properly paired

MultiQC over STAR logs:

```
multiqc $OUTDIR/ -o qc_post_align/ -n alignment_qc
```

MultiQC will parse all _Log.final.out files and produce a comparative bar chart of mapping rate categories across samples. Mapping rates should be similar if RNA quality was consistent.

Generate count matrix with featureCounts

```
BAM_LIST=$(ls ${OUTDIR}/*Aligned.sortedByCoord.out.bam | tr '\n' ' ' )

featureCounts \
  -a $GTF \
  -o counts/count_matrix_raw.txt \
  -T 8 \
  -p --countReadPairs \
  -s 2 \
  -t exon \
  -g gene_id \
  $BAM_LIST

# Extract just the count columns (remove featureCounts header metadata)
tail -n +2 counts/count_matrix_raw.txt | \
  cut -f1,7- > counts/count_matrix_clean.txt

# Number of genes with non-zero counts
awk 'NR>1{for(i=2;i<=NF;i++) if($i>0) {print; break}}' counts/count_matrix_clean.txt | wc -l

# Total assigned reads per sample (from featureCounts summary file)
cat counts/count_matrix_raw.txt.summary

# Quick look at top expressed genes
sort -k2 -rn counts/count_matrix_clean.txt | head -20
```

Category	Meaning	Acceptable range
Assigned	Reads successfully counted to a gene	>60%
Unassigned_NoFeatures	Mapped to genome but not an annotated exon (introns, intergenic)	20–40% for RNA-seq
Unassigned_Ambiguity	Overlapped >1 gene	<5%
Unassigned_MultiMapping	NH > 1, multi-mappers excluded	5–15%
Unassigned_Unmapped	Read failed to align	should be ~0 at this stage

High Unassigned_NoFeatures is normal for RNA-seq -mammalian genes have large introns and featureCounts only counts exonic reads. If >50%, verify strandedness setting first.

DESeq2 Workflow, Statistics & Biological Interpretation

1.1 Experimental design

DESeq2 operates on a design formula that tells the model which biological variables explain variation in gene expression. Everything downstream - normalization, dispersion estimation, hypothesis testing - flows from this.

example

Condition: Control (0 Gy) vs. 4 Gy irradiated
Replicates: 2 per condition (minimum; power is low - interpret with caution)

Design formula: ~ condition

The design formula ~ condition means: model gene counts as a function of condition (Control vs. 4 Gy). The intercept is the reference level (Control by default).

DESeq2 estimates within-group variance (biological variability) from replicates. With n=2 per group, variance estimation is very imprecise - DESeq2 borrows information across all genes (empirical Bayes shrinkage) to compensate, but statistical power remains limited.

If samples were processed in two batches, or animals differ in age/sex:

```
~ batch + condition      # control for batch
~ sex + condition        # control for sex
~ batch + sex + condition
```

The last variable in the formula is the one tested. Confounders not in the model inflate within-group variance and reduce power.

1.2 Why you cannot compare raw counts directly

- Sequencing depth differs between samples
- Gene length differs
- Compositional bias (the "library composition" problem)

This is the most counterintuitive and most important. If one gene becomes massively upregulated (e.g. stress response gene goes from 1% to 20% of all reads), every other gene's count will appear to go down - not because they are actually repressed, but because the total read count is fixed. This is why RPKM/FPKM fail for DE analysis and why DESeq2 uses its own normalization.

1.3 Normalization strategies - what exists and what DESeq2 uses

Method	Corrects Depth	Corrects Length	Suitable for DE
Raw counts	No	No	No
CPM (counts per million)	Yes	No	No
RPKM / FPKM	Yes	Yes	No - not comparable across samples
TPM	Yes	Yes	No - sum-constrained, compositional
DESeq2 size factors	Yes	No	Yes
TMM (edgeR)	Yes	No	Yes

DESeq2 does NOT correct for gene length. Length correction changes the relative proportions between genes within a sample, which would distort the between-sample comparison DESeq2 is designed to make. For DE testing, length correction is not needed.

TPM is appropriate for comparing expression levels of the same gene across samples in a descriptive sense but is not appropriate as input to DESeq2 - always use raw integer counts.

2.1 Size factor estimation - the median-of-ratios method

DESeq2 computes a size factor for each sample that accounts for both sequencing depth and library composition simultaneously.

1. For each gene, compute the geometric mean count across all samples (the "pseudo-reference sample")
2. For each sample, divide every gene's count by the geometric mean for that gene → this gives a ratio per gene per sample
3. The size factor for that sample = the median of all these ratios

The median is used (not mean) because it is robust to the few highly expressed genes that dominate the count distribution. Genes with a geometric mean of zero are excluded.

Two samples with different sequencing depths but the same library composition will have size factors proportional to their depths. A sample where one gene captured an unusual share of reads will have that gene's ratio as an outlier - ignored by the median.

```
sizeFactors(dds)
# Should be roughly centered on 1.0; values 0.5-2.0 are normal
# A size factor of 0.5 means this sample had roughly half as many usable reads
# after accounting for composition
```

2.2 Dispersion estimation

If gene counts followed a Poisson distribution, the variance would equal the mean. RNA-seq data is overdispersed - the variance is consistently larger than the mean, even for the same gene measured in identical biological replicates.

Source of overdispersion: true biological variability between individual animals. Even inbred mice of the same strain, same age, same treatment will not express Gene X at exactly the same level. This between-animal variability is on top of the Poisson counting noise.

DESeq2 uses the **negative binomial distribution**, which adds a dispersion parameter α to the Poisson:

$$\text{Var}(\text{count}) = \text{mean} + \alpha \times \text{mean}^2$$

Where α is the gene-specific dispersion. High α = high biological variability. Low α = gene is tightly regulated, counts are consistent across replicates.

The problem with n=2: you cannot estimate α reliably from 2 data points per gene. DESeq2 solves this with empirical Bayes shrinkage:

1. Estimate a rough per-gene dispersion from the data
2. Fit a trend line: dispersion as a function of mean count (high-expression genes tend to have lower dispersion)
3. Shrink each gene's estimated dispersion toward the trend - genes with little data are pulled strongly toward the trend; genes with more data are pulled less

This "borrowing strength" across all ~20,000 genes is what makes DESeq2 work reliably with small sample sizes, but it also means results with n=2 depend heavily on the assumption that the dispersion trend is correct.

That's why you have at least 3 replicates.

2.3 The DESeq2 statistical test

After normalization and dispersion estimation, DESeq2 fits a generalized linear model (GLM) with negative binomial errors for each gene:

$$\log_2(\text{normalized count}) = \text{intercept} + \beta \times \text{condition}$$

Where β = log2 fold change for the condition effect.

The **Wald test** evaluates whether $\beta \neq 0$:

- Compute Wald statistic = $\beta / \text{SE}(\beta)$
- Under the null hypothesis (no DE), this follows a standard normal distribution
- Convert to a two-tailed p-value

LRT (Likelihood Ratio Test) as an alternative: Used when comparing more than two conditions or when testing whether condition matters at all. Tests the full model vs. a reduced model that drops the condition term. More appropriate for complex designs with multiple timepoints or doses.

3.1 Log2 fold change and shrinkage

Raw log2 fold change = $\log_2(\text{mean_4Gy} / \text{mean_Control})$

The problem with raw LFC: genes with very low counts produce extreme and unreliable fold changes. A gene with 1 count in Control and 3 counts in 4 Gy shows a "3-fold change" - but this is statistically meaningless at such low counts.

DESeq2 applies **apeglm / ash** LFC shrinkage (`lfcShrink()`):

- Shrinks large LFC estimates for low-count genes toward zero
- High-count genes with genuine large fold changes are largely unaffected
- Makes volcano plots and ranked gene lists more interpretable
- **Always use shrunk LFC for visualization and ranking; use unshrunk LFC only if you need the raw estimate for a specific reason**

```
res_shrunk <- lfcShrink(dds, coef = "condition_4Gy_vs_Control", type = "apeglm")
```

3.2 Multiple testing - why a 0.05 p-value cutoff alone is wrong

You are testing ~20,000 genes simultaneously. At $p < 0.05$, you expect $0.05 \times 20,000 = 1,000$ false positives by chance alone, even if no gene is truly differentially expressed.

Benjamini-Hochberg (BH) FDR correction - what DESeq2 applies:

1. Rank all genes by p-value (smallest to largest)
2. For rank i out of m total tests, adjusted p-value = $p \times m / i$
3. The `padj` column in DESeq2 results is the BH-adjusted p-value, interpretable as the False Discovery Rate

Standard thresholds:

- `padj < 0.05` - at most 5% of called DEGs are expected to be false positives
- `padj < 0.1` - more permissive; acceptable for exploratory analysis or small sample sizes
- `|log2FC| > 1` - at least 2-fold change; filters for biologically meaningful magnitude

Independent filtering: DESeq2 automatically sets `padj = NA` for genes with very low mean counts across all samples. These genes have so little power that including them in multiple testing correction would waste the FDR budget. Genes with `padj = NA` were filtered out; they are not errors.

Import count matrix

```
library(DESeq2)
library(tidyverse)

# Read count matrix from featureCounts output
counts_raw <- read.table("counts/count_matrix_clean.txt",
                        header = TRUE,
                        row.names = 1,
                        sep = "\t")

# Inspect
dim(counts_raw)          # expect ~55,000 genes × 4 samples (mouse GENCODE)
head(counts_raw)
colSums(counts_raw)      # total counts per sample - check for major imbalances
summary(rowSums(counts_raw) == 0) # how many genes have zero counts across all
samples?

# Column names come from BAM file names - rename them now to something readable
# ~30–40% of genes will have zero counts in any given tissue - this is normal
# Very large imbalance in colSums (>3× difference) suggests a problematic sample

# Rename columns to meaningful sample names
colnames(counts_raw) <- c("Control_1", "Control_2", "4Gy_1", "4Gy_2")

# Keep only integer counts (featureCounts sometimes outputs with decimal from --
fraction)
counts_mat <- round(as.matrix(counts_raw))
```

Create DESeq2 object

```
# Sample metadata - must match column order of count matrix exactly
coldata <- data.frame(
  condition = factor(c("Control", "Control", "4Gy", "4Gy")),
  row.names = colnames(counts_mat))

# Set reference level - Control is baseline; 4Gy is compared to it
coldata$condition <- relevel(coldata$condition, ref = "Control")

# Create DESeqDataSet object
dds <- DESeqDataSetFromMatrix(
  countData = counts_mat,
  colData   = coldata,
  design    = ~ condition)

# Pre-filter: remove genes with <10 counts across all samples
# Rationale: these genes have no statistical power; filtering reduces multiple
testing burden
keep <- rowSums(counts(dds)) >= 10
dds <- dds[keep, ]

dim(dds) # how many genes remain after filtering?
```

filtering threshold: The cutoff of 10 is a common starting point but not fixed. The DESeq2 vignette recommends at least 1 count per sample on average, which for 4 samples = 4 total. Using 10 is slightly stricter and reduces noise. For very lowly expressed genes in a 2-replicate experiment, you have essentially no power anyway.

Run DESeq2

```
# Run the full DESeq2 pipeline in one call:
```

```
# 1. Estimate size factors
# 2. Estimate dispersions
# 3. Fit negative binomial GLM
# 4. Wald tests for all coefficients
dds <- DESeq(dds)

# Inspect what was computed
resultsNames(dds) # shows available coefficients
# "Intercept" "condition_4Gy_vs_Control"

# Check size factors
sizeFactors(dds)

# Check dispersion - should see dots around a curve, some above (outliers),
# gene-est scattered, fitted trend line, final shrunken estimates
plotDispEsts(dds)
```

dispersion plot:

- Black dots = per-gene dispersion estimates (noisy, especially at low counts)
- Red curve = fitted trend (dispersion decreases as mean count increases - this is expected)
- Blue dots = final shrunken estimates used in testing (pulled toward red curve)
- Circled dots = genes whose estimates were not shrunk (outliers flagged by DESeq2)
- If the red curve is flat or upward-sloping: suggests batch effects or problematic samples

Extract results and DEG list

```
# Extract results for condition: 4Gy vs Control
res <- results(dds,
               name = "condition_4Gy_vs_Control",
               alpha = 0.05) # alpha sets the threshold for independent
filtering

summary(res)
# Shows: number upregulated, downregulated, low-count filtered, outliers

# Apply LFC shrinkage for visualization and ranking
res_shrunk <- lfcShrink(dds,
                       coef = "condition_4Gy_vs_Control",
                       type = "apeglm")

# Convert to data frame and add gene IDs
res_df <- as.data.frame(res_shrunk) %>%
  rownames_to_column("gene_id") %>%
  arrange(padj)

head(res_df, 20)
```

- **baseMean** - mean normalized count across all samples; low values = low-expressed gene
- **log2FoldChange** - shrunken LFC; positive = higher in 4Gy; negative = lower in 4Gy
- **lfcSE** - standard error of the LFC estimate
- **stat** - Wald statistic = LFC / lfcSE
- **pvalue** - raw p-value from Wald test
- **padj** - BH-adjusted p-value; use this for significance calls; NA = filtered out

```
# Define DEGs
DEGs <- res_df %>%
  filter(padj < 0.05, abs(log2FoldChange) > 1)
```

```
nrow(DEGs)
table(DEGs$log2FoldChange > 0) # upregulated vs downregulated counts

# Write to file
write.csv(res_df, "results/DESeq2_all_genes.csv", row.names = FALSE)
write.csv(DEGs, "results/DESeq2_DEGs_padj05_lfc1.csv", row.names = FALSE)
```

4.1 gene lists to biology

Pathway / functional enrichment: Which biological processes are overrepresented in your DEG list? Uses tools like clusterProfiler (GO, KEGG, Reactome).

is the DEG list enriched for DNA repair genes, cell cycle arrest, apoptosis, immune activation?

Network and regulatory context: Which transcription factors or upstream regulators could explain the observed pattern? Tools: GSEA (gene set enrichment), dorothea (TF activity), PROGENy (pathway activity scores).

Common interpretation pitfalls:

- Confusing statistical significance with biological importance - a gene with padj = 1e-50 may have a fold change of 1.1 (biologically trivial)
- Interpreting direction of change in isolation - check which direction makes biological sense for your model

PCA plot

PCA reduces the high-dimensional gene expression space (one axis per gene) to a small number of principal components that capture the most variance. PC1 captures the most variance, PC2 the second most. Samples that are transcriptomically similar cluster together.

- PC1 should separate Control from 4Gy if irradiation has a strong transcriptomic effect
- Replicates within a group should be close together
- Outlier samples (one replicate far from its partner) = problematic sample; investigate before proceeding

```
library(ggplot2)

# Variance-stabilizing transformation for PCA (not for DE testing - just for
# visualization)
# VST is preferred over rlog for n > 30 samples; both are fine for small n
vsd <- vst(dds, blind = TRUE)
# blind = TRUE: transformation ignores the design - appropriate for
# QC/exploration

# Built-in DESeq2 PCA
plotPCA(vsd, intgroup = "condition")

# Custom ggplot2 version with cleaner aesthetics
pca_data <- plotPCA(vsd, intgroup = "condition", returnData = TRUE)
pct_var <- round(100 * attr(pca_data, "percentVar"), 1)

ggplot(pca_data, aes(x = PC1, y = PC2, color = condition, label = name)) +
  geom_point(size = 4) +
  ggrepel::geom_text_repel(size = 3.5) +
  xlab(paste0("PC1: ", pct_var[1], "% variance")) +
  ylab(paste0("PC2: ", pct_var[2], "% variance")) +
  theme_bw() +
  scale_color_manual(values = c("Control" = "#2166AC", "4Gy" = "#D6604D"))
```

- If <30%, condition is not the dominant source of variation - something else (batch, animal-to-animal variation) may dominate
- If replicates are spread apart on PC2, this may represent individual animal variability - normal in vivo

Volcano plot

```
library(ggplot2)
library(ggrepel)

# Add significance classification
volcano_df <- res_df %>%
  mutate(
    significance = case_when(
      padj < 0.05 & log2FoldChange > 1 ~ "Up in 4Gy",
      padj < 0.05 & log2FoldChange < -1 ~ "Down in 4Gy",
      TRUE ~ "NS"),
    neg_log10_padj = -log10(padj))

# Label top 15 genes by padj
top_genes <- volcano_df %>%
  filter(significance != "NS") %>%
  slice_min(padj, n = 15)

ggplot(volcano_df, aes(x = log2FoldChange, y = neg_log10_padj, color =
significance)) +
  geom_point(alpha = 0.6, size = 1.2) +
  geom_text_repel(data = top_genes, aes(label = gene_id),
    size = 3, max.overlaps = 20) +
  geom_hline(yintercept = -log10(0.05), linetype = "dashed", color = "grey40") +
  geom_vline(xintercept = c(-1, 1), linetype = "dashed", color = "grey40") +
  scale_color_manual(values = c("Up in 4Gy" = "#D6604D",
    "Down in 4Gy" = "#2166AC",
    "NS" = "grey70")) +
  labs(x = "log2 Fold Change (4Gy / Control)",
    y = "-log10 adjusted p-value",
    color = "Direction") +
  theme_bw()
```

Heatmap

Expression of the top DEGs across all samples, with hierarchical clustering to reveal groups of co-regulated genes and sample similarity. Useful for confirming that the pattern of DE is consistent across replicates.

```
library(pheatmap)
library(RColorBrewer)

# Select top 50 DEGs by padj for heatmap
top50 <- DEGs %>%
  slice_min(padj, n = 50) %>%
  pull(gene_id)

# Extract VST-normalized counts for these genes
mat <- assay(vsd)[top50, ]

# Scale by row (z-score per gene) - allows color to reflect relative change
# without being dominated by absolute expression level
mat_scaled <- t(scale(t(mat)))

# Annotation for columns
col_anno <- data.frame(
```

```
Condition = coldata$condition,
row.names = rownames(coldata))

ann_colors <- list(
  Condition = c(Control = "#2166AC", "4Gy" = "#D6604D"))

pheatmap(mat_scaled,
  annotation_col = col_anno,
  annotation_colors = ann_colors,
  color = colorRampPalette(rev(brewer.pal(11, "RdBu")))(100),
  cluster_rows = TRUE,
  cluster_cols = TRUE,
  show_rownames = TRUE,
  show_colnames = TRUE,
  fontsize_row = 7,
  breaks = seq(-2, 2, length.out = 101),
  main = "Top 50 DEGs - 4Gy vs Control (z-score)")
```